RAPPORT INTERMÉDIAIRE

Nom du projet
Amélioration et évolution d'un site open source de récolte de données expérimentales

Auteurs, date & version du document
Équipe projet : Elouan Dubois, Pablo Pierre, Mathis Gillaizeau
Date : 7 janvier 2026

Table des matières

Table des matières..... 0
Introduction..... 1
1 - Présentation de l'entreprise et du sujet..... 1
Histoire de l'entreprise..... 1
Histoire du projet..... 2
Présentation de l'équipe MOA..... 3
Présentation de l'équipe MOE..... 3
Coordonnées des parties du projet..... 3
Les objectifs..... 4
2 - Expression de besoins + planning..... 6
Modèle du domaine..... 6
Besoins Fonctionnels..... 7
Description des acteurs / utilisateurs..... 7
Liste hiérarchisée des besoins (méthode MoSCoW)..... 8
Besoins non fonctionnels..... 10
Evolution potentielle des besoins..... 11
Risques..... 11
Liste des risques potentiels..... 11
Tableau des probabilités et conséquences de ces risques..... 12
Politique de gestion des risques..... 12
Etat initial du projet..... 13
Planning global des sprints..... 15
3 - Revue de littérature scientifique et technologique..... 29
Expérience utilisateur et qualité des données expérimentales..... 29
Conception de formulaires interactifs en contexte expérimental..... 29
Oculométrie et analyse de l'interaction utilisateur..... 30

Collecte de données numériques et biais méthodologiques..... 30
Sécurité, éthique et protection des données expérimentales..... 31
Logiciels open source et recherche expérimentale..... 31
Architectures web modernes pour applications expérimentales.....32
Outils de développement collaboratif et gestion des versions..... 32
Internationalisation des interfaces (i18n)..... 33
4 - Bibliographie..... 33
5 - Conclusion.....34

Introduction

Depuis le mois de novembre, le travail de développement s'est concentré sur l'intégration des évolutions fonctionnelles et techniques demandées par le client. Cette nouvelle version de l'application vise principalement à améliorer l'ergonomie, l'organisation des données et la productivité des utilisateurs.

Parmi les principaux changements, un système de groupes a été mis en place afin de structurer les formulaires et de simplifier leur gestion (recherche, exportation et importation). La page d'accueil a également été repensée, avec une interface plus claire et un regroupement des actions plus intuitif.

Enfin, plusieurs fonctionnalités utilitaires, telles que la duplication de composants, ont été ajoutées afin de faciliter et d'accélérer les usages quotidiens.

Ce rapport intermédiaire présente l'état d'avancement du projet après plusieurs itérations de développement, en s'appuyant sur le rapport initial tout en intégrant les évolutions fonctionnelles et techniques réalisées depuis.

1 - Présentation de l'entreprise et du sujet

Histoire de l'entreprise

XP-LAB est une plateforme de recherche de l'Université de Nantes, située à la Halle 6 Ouest, spécialisée dans la recherche expérimentale et l'analyse des comportements humains. Dans ses activités, la plateforme utilise des outils numériques pour concevoir, déployer des protocoles expérimentaux et analyser les résultats récupérés à l'issue des expérimentations les suivant, notamment à travers des formulaires interactifs.

La plateforme dispose d'espaces spécialisés (cabines, salle immersive, régie…) et d'équipements de pointe (oculométrie, EEG, capteurs, VR…) pour mener des études sur l'interaction, la perception ou la cognition.

Dans une logique d'ouverture scientifique, XP-LAB s'appuie sur des solutions open source pour développer ses outils, afin qu'ils soient réutilisables par d'autres chercheurs et laboratoires, en France comme à l'international.

Enfin, XP-LAB fait partie d'un réseau régional de UserLabs et travaille avec des partenaires publics et privés pour faire le lien entre recherche académique et innovation.

Histoire du projet

Le projet sur lequel nous intervenons est un site open source de récolte de données expérimentales. Celui-ci permet de créer, déployer et administrer des questionnaires complexes, avec une gestion multi-formulaires et des outils de suivi.

Le dispositif global repose sur l'articulation de trois outils principaux : Tobii Pro Lab, OpenSesame et le site open source de récolte de données sur lequel nous intervenons.
Tobii Pro Lab est un logiciel d'oculométrie utilisé pour enregistrer les mouvements oculaires des participants pendant une expérience. OpenSesame, quant à lui, sert à concevoir et contrôler le déroulement des expériences comportementales (affichage de stimuli, enregistrement des réponses, etc.).

Ce projet a été créé car les logiciels utilisés pour les expériences manquent d'outils de questionnaire fiable, facile d'utilisation et ergonomique. Le logiciel Tobii Prolab n'a pas la possibilité de créer des questionnaires, ce qui est embêtant pour un logiciel servant à des expériences d'oculométrie car ça oblige le cobaye à détourner son attention du logiciel afin de répondre au questionnaire qui serait donc sur une autre plateforme. Et pour le logiciel OpenSesame, les fonctionnalités déjà présentes, développées par le groupe précédent, permettant la création de questionnaire, sont très limitées et très peu agréables à utiliser.

Le site web développé par l'équipe de l'an dernier vient compléter ces outils en assurant la collecte structurée des réponses des participants via des formulaires personnalisables. Il permet de créer des questionnaires en ligne, de les intégrer dans les expériences (via Tobii Pro Lab ou un futur plugin OpenSesame), puis de récupérer les données pour analyse.

Le schéma d'utilisation typique est le suivant :

1. Le chercheur conçoit un protocole expérimental dans OpenSesame ou Tobii Pro Lab.
2. Il crée un formulaire de recueil de données sur le site (questionnaire pré-expérience, post-expérience, ou intégré).
3. Le participant interagit avec ce formulaire pendant ou après l'expérience.

4. Les réponses sont stockées sur la plateforme et exportées au format compatible pour analyse statistique ou croisement avec les données comportementales collectées.

Cependant, la version actuelle du site présente plusieurs limites ergonomiques et fonctionnelles. L'objectif du projet est donc d'améliorer l'outil existant, tant sur le plan technique que sur celui de l'expérience utilisateur, afin de répondre aux besoins concrets des chercheurs, ainsi que le rendre compatible avec ces deux logiciels.

Présentation de l'équipe MOA

L'équipe est composée d'un responsable opérationnel et d'un responsable scientifique de la plateforme, travaillant dans la Halle 6 de l'université de Nantes. Le projet s'inscrit dans la stratégie du XP-LAB, qui vise à renforcer ses capacités d'expérimentation numérique tout en diffusant des outils open source au service de la recherche. L'enjeu principal est de moderniser un site de récolte de données expérimentales afin de le rendre plus ergonomique, stable et performant. Ce projet répond à la volonté de la plateforme d'améliorer la fiabilité des données collectées, de simplifier la conception des formulaires et de favoriser la réutilisation des outils par d'autres personnes. Il s'agit également de garantir une meilleure maintenance technique et une interopérabilité accrue avec Tobii pro lab.

Présentation de l'équipe MOE

L'équipe est composée d'un enseignant chercheur, prenant ici le rôle de tuteur pédagogique, et de trois étudiants de quatrième année de la spécialité informatique de Polytech Nantes.

Coordonnées des parties du projet

MOA (Maîtrise d'Ouvrage – XP-LAB)

- **Qui :** François Bouffard
  - **Rôle :** Responsable Opérationnel Userlab
  - **Mail :** Francois.Bouffard@univ-nantes.fr
  - **Mobile :** 02 53 00 80 04

- **Qui :** Matthieu Perreira Da Silva
  - **Rôle :** Responsable scientifique du Userlab
  - **Mail :** Matthieu.Perreiradasilva@univ-nantes.fr
  - **Mobile :** 02 40 68 30 60

MOE (Maîtrise d'Œuvre – Équipe projet)

- **Olivier Aubert**
  - **Rôle :** Tuteur pédagogique
  - **Mail :** Olivier.Aubert@univ-nantes.fr
  - **Mobile :** 02 40 68 31 46

- **Elouan Dubois**
  - **Rôle :** Développeur
  - **Mail :** elouan.dubois@etu.univ-nantes.fr
  - **Mobile :** 06 69 90 75 99

- **Pablo Pierre**
  - **Rôle :** Développeur
  - **Mail :** pablo.pierre@etu.univ-nantes.fr
  - **Mobile :** 07 69 03 07 97

- **Mathis Gillaizeau**
  - **Rôle :** Chef de projet / développeur
  - **Mail :** mathis.gillaizeau@etu.univ-nantes.fr
  - **Mobile :** 07 86 31 28 60

Les objectifs

Nous allons principalement nous concentrer sur la partie site et compatibilité avec Tobii Prolab. L'intégration avec OpenSesame sera abordée de manière exploratoire, sous réserve du temps disponible.

La solution proposée permettra :

- d'améliorer l'ergonomie générale de l'éditeur de formulaires, pour faciliter la conception et la modification des expériences ;

- de renforcer les possibilités de personnalisation (CSS, JSON, hiérarchies de projets) afin d'adapter l'outil aux contextes variés de recherche ;

- de proposer des outils de monitoring (statistiques, suivi des réponses, hiérarchies) afin d'assurer un meilleur contrôle sur les données collectées ;

- de faciliter l'I/E des données, le déploiement du site, pour permettre une utilisation collaborative entre chercheurs et laboratoires ;

- de préparer l'outil à une utilisation internationale (textes multilingues, paramètres flexibles).

- d'assurer un déploiement technique stable et robuste (build, serveur unique, optimisation des performances) ;

- d'ajouter des fonctionnalités avancées comme la conditionnalité du passage de pages, le copier-coller de composants ou la création de composants personnalisés ;

- de prévoir une intégration future avec la plateforme OpenSesame pour des usages élargis en recherche expérimentale.

2 - Expression de besoins + planning

Diagramme UML du projet

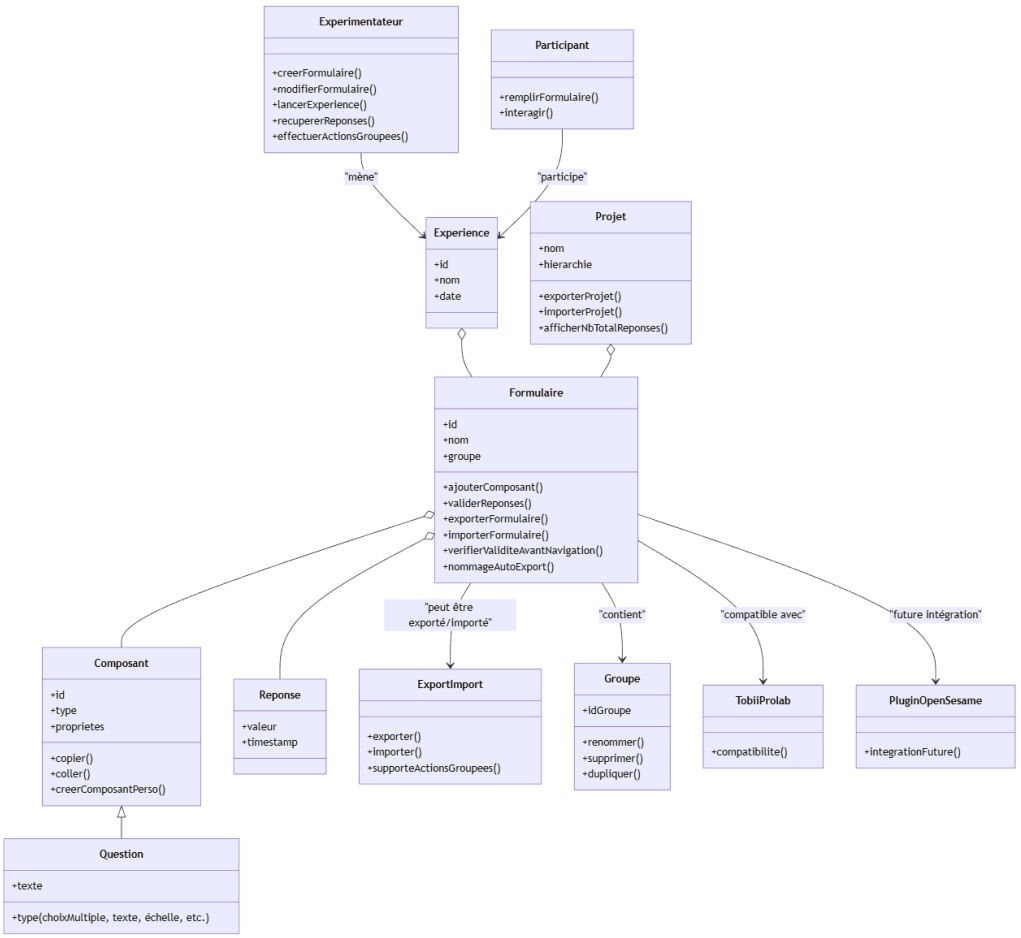


Figure 1 - Diagramme UML du projet

Expérimentateur : Personne responsable de la conception et du suivi des expériences. Il crée, configure et déploie les formulaires utilisés pour collecter des données auprès des participants, puis récupère et analyse les réponses.

Participant : Individu prenant part à une expérience. Il interagit avec les formulaires en répondant aux questions posées et en fournissant des données pour l'analyse de l'expérimentateur.

Expérience : Ensemble structuré d'activités ou d'étapes visant à observer, mesurer ou tester un phénomène ou une hypothèse scientifique. Elle inclut généralement la présentation de formulaires et la collecte de réponses.

Formulaire : Document ou interface numérique permettant de structurer les questions posées aux participants. Un formulaire peut être composé de plusieurs composants et contient les réponses collectées.

Composant : Élément constitutif d'un formulaire. Il définit un bloc fonctionnel pouvant être un champ de saisie, un bouton, une image ou un autre type d'élément interactif.

Question : Type spécifique de composant destiné à recueillir une réponse du participant. Elle peut être de type texte libre, choix unique, choix multiples, échelle, date/heure, etc.

Réponse : Donnée fournie par un participant à une question ou un composant du formulaire. Elle est enregistrée avec un identifiant et éventuellement un timestamp.

Projet : Ensemble regroupant plusieurs formulaires et expériences, permettant d'organiser le travail de recherche et de gérer la hiérarchie des formulaires et des expériences.

ExportImport : Module ou fonctionnalité permettant d'exporter ou d'importer des formulaires, des projets ou des réponses afin de les partager ou de les réutiliser dans différents contextes ou systèmes.

TobiiProlab : Logiciel tiers spécialisé dans l'oculométrie. Le système doit pouvoir s'y connecter pour récupérer ou synchroniser les données collectées via les formulaires.

PluginOpenSesame : Extension prévue pour intégrer le site avec le logiciel OpenSesame, permettant aux formulaires de s'afficher et de fonctionner directement dans les expériences gérées par OpenSesame.

Besoins Fonctionnels

Description des acteurs / utilisateurs

Le principal utilisateur du site est le responsable opérationnel du userlab ou des chercheurs dans le domaine. Ces utilisateurs ont un niveau technique intermédiaire : ils maîtrisent les outils informatiques et expérimentaux, mais ne sont pas nécessairement des développeurs. Leur objectif est de créer, configurer et administrer des formulaires pour collecter des données expérimentales.

Un second type d'utilisateur est le chercheur, qui ne dispose pas nécessairement de compétences techniques avancées. Il doit néanmoins pouvoir utiliser l'outil de manière intuitive, sans connaissance préalable du développement web.

Un troisième type d'utilisateur est l'administrateur technique. Celui-ci dispose d'une expérience plus avancée en programmation et en déploiement web. Il assure la maintenance, les mises à jour et le support de la plateforme.

Enfin, les participants aux expériences constituent un troisième profil : ils interagissent uniquement avec les formulaires créés, via une interface simple et intuitive. Leur usage se limite à répondre aux questionnaires.

Liste hiérarchisée des besoins (méthode MoSCoW)

Must have	Should have
1 - Afficher le nombre total de réponses sur la page d'accueil. 2 - Permettre la sélection multiple de formulaires ou groupes pour effectuer des actions groupées. 3 - Nommer automatiquement les fichiers exportés d'après le nom du formulaire. 4 - Autoriser le passage à la page suivante uniquement si les réponses sont valides. 5 - Permettre le copier-coller de composants. 6 - Permettre la création de composants personnalisés ou la génération d'un composant à partir d'un ensemble existant 7 - Offrir la possibilité d'exporter et importer un formulaire ou groupe	8 - Permettre de modifier le nom d'un formulaire directement depuis l'éditeur sans quitter la page. 9 - Repenser l'ergonomie de certaines interactions, notamment la navigation entre les pages de formulaires et l'accès aux paramètres, afin de rendre la conception plus fluide et intuitive. 10 - Autoriser la modification du CSS (soit via un fichier, soit via une interface dédiée). 11 - Créer une hiérarchie de formulaires (dossiers) avec export global. 12 - Restreindre la pagination à certaines pages spécifiques. 13 - Ajouter des outils de suivi des questionnaires (ex. nombre de réponses, progression). 14 - Configurer le client pour un déploiement unique (sans serveur de dev React séparé).

Could have	Won’t have
15 - Cacher l’identifiant du formulaire sur la page d’accueil. 16 - Ajouter un bouton pour passer rapidement à l’ID utilisateur suivant. 17 - Permettre l’édition ou la visualisation du JSON du formulaire en cas de problème. 18 - Ajouter une variable indiquant le numéro de page dans les formulaires multipages. 19 - Gérer les paramètres passés dans l’URL du formulaire. 20 - Mettre en place l’internationalisation (traduction des textes).	21 - Développement complet du plugin OpenSesame pour l’intégration directe du questionnaire au sein du logiciel expérimental. 22 - Proposer à Form-js d’y ajouter la duplication des composants

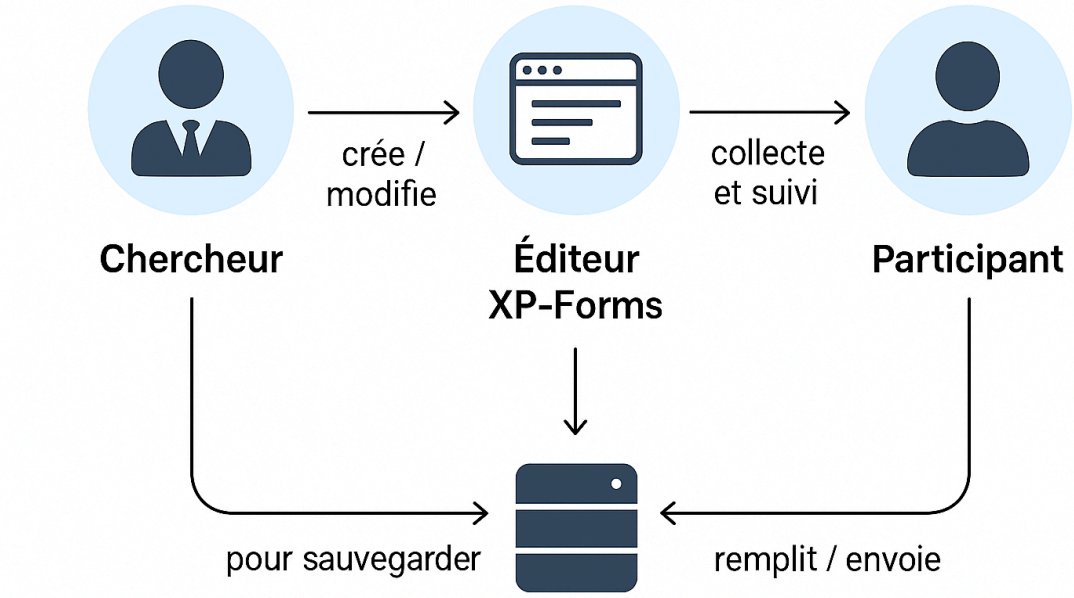


Figure 2 - Schéma du modèle du domaine

Besoins non fonctionnels

Compatibilité

- Le logiciel doit être développé et testé principalement sous Windows, tout en assurant un fonctionnement correct sur d'autres systèmes si nécessaire.
- Le frontend doit être fonctionnel sur les navigateurs les plus utilisés, comme google chrome, firefox, safari, etc…
- Le backend doit être fonctionnel sur Windows, Linux, MacOS, etc…

- Les formulaires doivent pouvoir être intégrés à Tobii ProLab pour les expérimentations.

- Le système doit rester compatible avec les dispositifs utilisés par l'XP-LAB, notamment les oculomètres.

Performance

- Les formulaires doivent se charger rapidement pour garantir une utilisation fluide.

Capacité

- Le système (backend) doit disposer d'une capacité de stockage suffisante pour stocker toutes les données saisies par les participants.

Extensibilité

- Le logiciel doit pouvoir s'adapter à l'augmentation progressive du nombre de formulaires et d'expériences, sans compromettre les performances ou la stabilité.

Maintenabilité

- Le code doit être clair et structuré, respectant les conventions de codage, afin de faciliter les modifications et évolutions futures.

- Une documentation claire et structurée doit accompagner le code livré, afin de faciliter la prise en main par de futurs développeurs et simplifier l'ajout de nouvelles fonctionnalités.

Ergonomie

- L’interface doit être intuitive et facile à utiliser pour les utilisateurs lors de la création des formulaires et pour les participants lors de leur remplissage.

Evolution potentielle des besoins

1. Un module de représentation graphique des résultats, permettant aux utilisateurs d’analyser rapidement les réponses collectées, par exemple sous forme d’histogrammes, de corrélations ou de courbes temporelles.

2. Un système d’authentification et de gestion des comptes expérimentateurs, renforçant la sécurité et permettant d’attribuer clairement les formulaires et résultats à chaque utilisateur, avec la possibilité de transférer des formulaires entre utilisateurs.

3. Des options supplémentaires pour les formulaires, telles que le choix d’un thème visuel clair ou sombre, l’affichage d’une barre de progression, ou la sélection de la langue du questionnaire pour faciliter une utilisation internationale.

4. Des widgets avancés pour enrichir les formulaires, par exemple des champs pour télécharger des fichiers, des curseurs de type input range, ou des visualisations intégrées directement dans le formulaire.

Risques

Liste des risques potentiels

(R1) Risques liés au contexte général et à l’entreprise :

- **Risque lié à l’utilisation de solutions externes**

    - Dépendance à des composants tiers qui peuvent évoluer ou ne plus être maintenus.

    - Problèmes de sécurité potentiels dus à des vulnérabilités présentes dans certaines dépendances logicielles utilisées par le projet, qu’elles soient open source ou propriétaires.

(R2) Risques techniques

- **Risque d’instabilité du site ou du serveur**

    - Problèmes techniques pouvant interrompre la collecte de données lors des expériences.

(R3) Risques organisationnels et humains

- **Risque lié à la gestion du projet**

    - Retards dans le développement pour cause de changement d'exigences, ou encore de mauvaise évaluation du temps de chaque tâche.

    - Manque de ressources (temps, compétences, personnel) pour couvrir toutes les fonctionnalités prévues.

(R4) Risques liés à la sécurité

- **Risque de vulnérabilités du système**

    - Possibilité d'attaques informatiques (injection, phishing, etc.) sur le site.

(R5) Risques spécifiques au projet

- **Risque de défaillance dans le suivi et le monitoring des réponses**

    - Données incomplètes ou erronées dues à des erreurs dans les outils de suivi et statistiques.
    - Perte de données lors de la collecte dûe à un problème serveur, à un bug du frontend ou du backend.

Tableau des probabilités et conséquences de ces risques

	Improbable	Peu probable	Probable	Très probable
Désastreux				
Sévère				
Modéré		R2/R4/R5		
Faible		R1	R3	

Politique de gestion des risques

R1 : Utiliser des composants stables et bien documentés pour limiter les risques liés aux dépendances.

R2 : Effectuer des tests réguliers pour s'assurer de la stabilité du site et du serveur.

R3 : Maintenir une régularité dans le suivi du projet et assurer une bonne communication entre les membres de l'équipe.

R4 : Mettre en place des mesures de sécurité standard.

R5 : Effectuer des vérifications régulières et des tests pour garantir la fiabilité du suivi et du monitoring des réponses.

Etat initial du projet

A l'état initial, le projet affichait un tableau des différents formulaires enregistrés (figure 3) et permettait d'effectuer des actions sur ces formulaires (voir, modifier, afficher les réponses, dupliquer et supprimer) un pop-up de confirmation (figure 6) apparaissait et nécessitait confirmation pour de nombreuses actions. Il y avait également la possibilité de créer des formulaires en assemblant différents composants les uns avec les autres (figure 4). Dans cette version du projet, il était aussi possible de répondre aux formulaires (figure 5) et d'enregistrer les réponses. Néanmoins, cette version avait quelques problèmes. En effet, elle ne permettait pas d'afficher le nombre de réponses pour chaque formulaire, et elle ne permettait pas d'effectuer des actions groupées, ce qui devient rapidement long et fastidieux dans le cas où il faut supprimer ou dupliquer de nombreux formulaires. Les pop up étaient aussi un problème, il y en avait trop et la plupart n'étaient pas utiles, ce qui impliquait simplement une perte de temps.

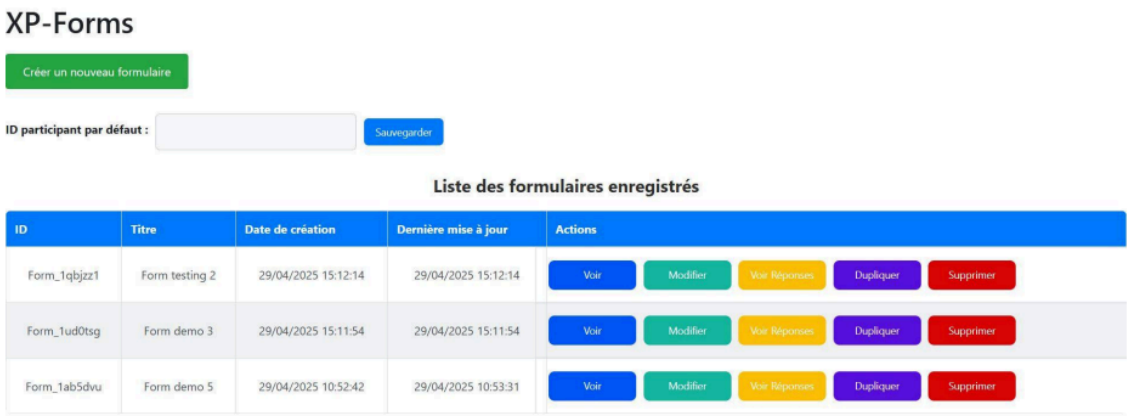


Figure 3 - Page d'accueil à l'état initial

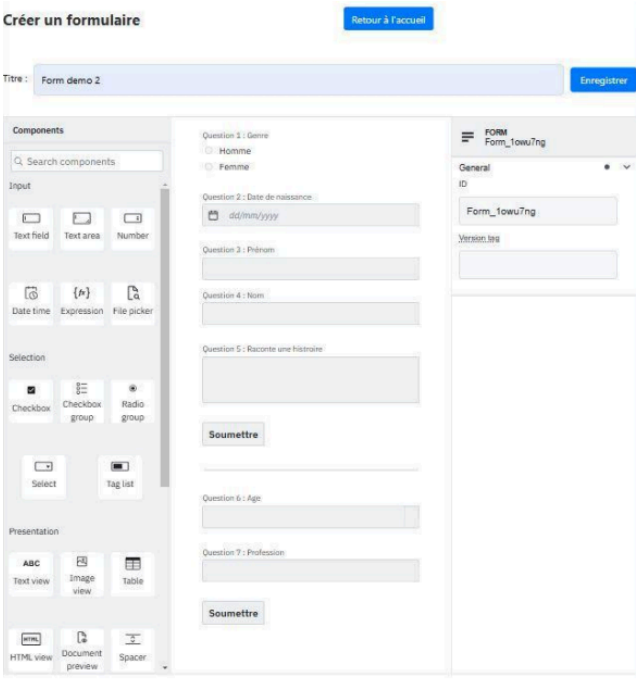


Figure 4 - Page d'édition de formulaires à l'état initial

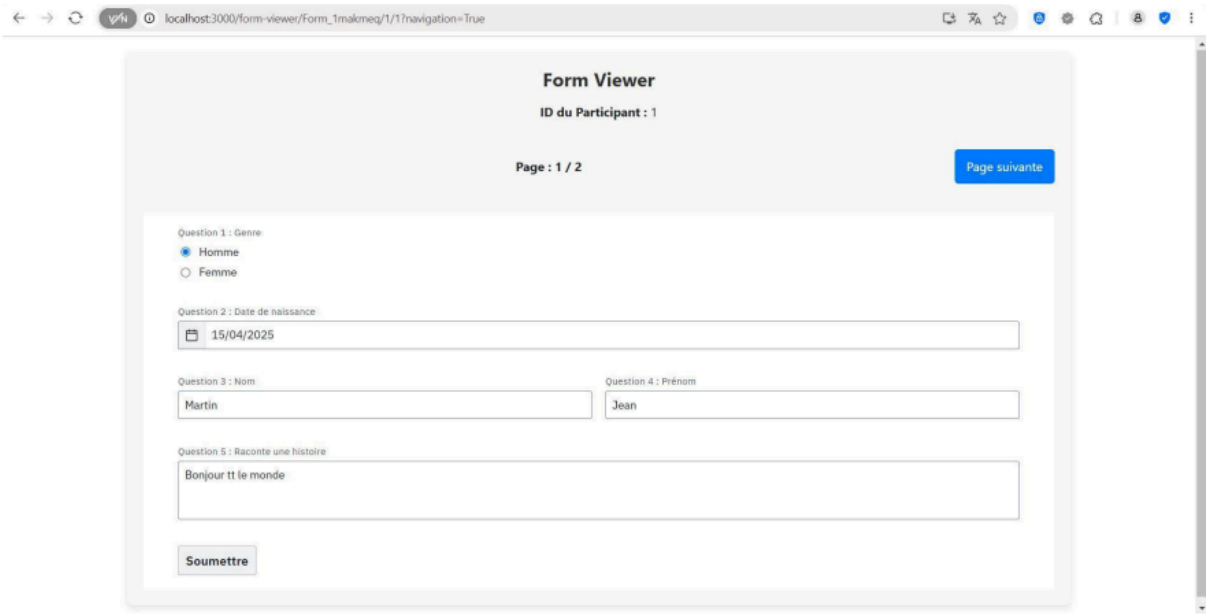


Figure 5 - Page de réponse au formulaire

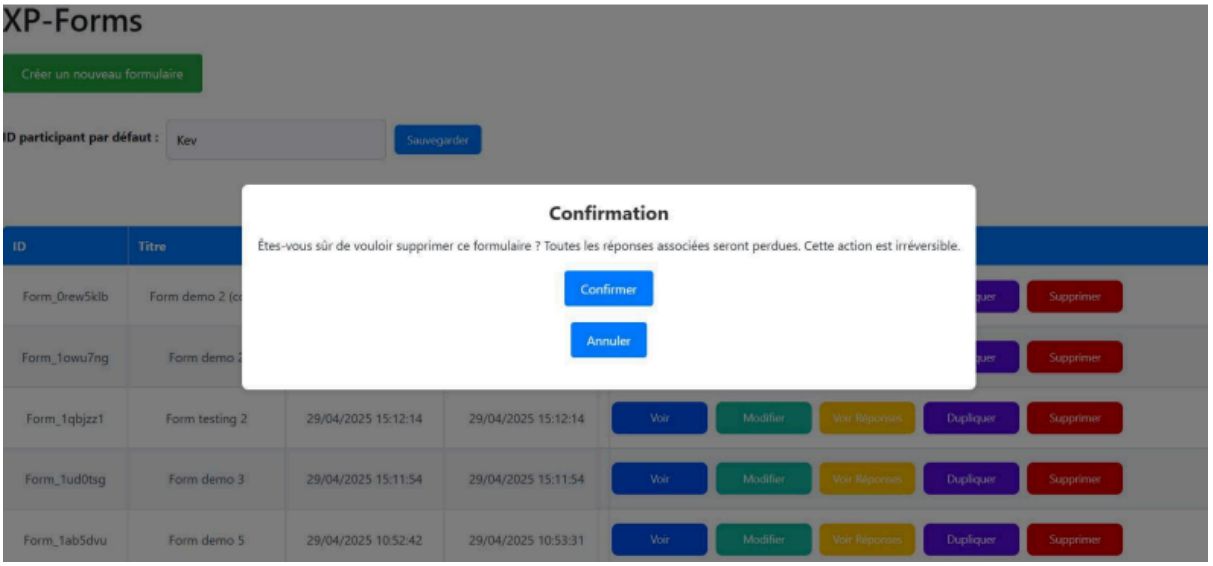


Figure 6 - Pop up de confirmation

Planning global des sprints

Planning réalisé

- **Itération 1 -** Commencer à coder des tâches simples côté front-end pour comprendre le fonctionnement du code :
    - Afficher le nombre total de réponses sur la page d'accueil
    - Permettre la sélection multiple de formulaires pour effectuer des actions groupées
    - Nommer automatiquement les fichiers exportés d'après le nom du formulaire
    - Autoriser le passage à la page suivante uniquement si les réponses sont valides
    - Cacher l'identifiant du formulaire sur la page d'accueil

Pour ce qui est des réponses, on a fait en sorte que chaque formulaire affiche le nombre de réponses reçues dans le tableau des formulaires sur la page d'accueil. Cela permet de voir rapidement le nombre de réponses sans l'ouvrir.

En ce qui concerne la fonctionnalité de la désactivation de la page suivante sur les formulaires, nous avons mis en place un système qui bloque la navigation tant que tous les champs obligatoires de la page ne sont pas remplis. A chaque modification de réponses (fromData), il y a la fonction validateCurrentPage() qui vérifie si tous les champs obligatoires sont complétés. Si un champ obligatoire est vide alors le bouton page suivante reste désactivé et une pop up s'affiche (ce qu'on modifiera lors de la prochaine itération).

Également nous avons mis en place des actions groupées, il suffit à l'utilisateur de sélectionner plusieurs formulaires et il peut les supprimer, dupliquer ou déplacer vers un autre groupe, ce principe de sélection rend beaucoup plus rapide et pratique la gestion des

formulaires.

On a aussi caché l'identifiant des formulaires sur la page d'accueil, cela permet de se concentrer sur les éléments essentiels du formulaire, maintenant l'utilisateur peut voir uniquement le titre, la date de création, la date de la dernière mise à jour et le nombre de réponses.

De même, nous avons fait en sorte que lors de l'exportation des réponses, les fichiers prennent automatiquement le nom du formulaire correspondant, ce qui simplifie l'exportation des données.



Figure 7 - Page d'accueil après itération 1

- **Itération 2 -** Commencer à coder des tâches plus complexes en codant côté back-end et revoir l'interface global de la page d'accueil :
  - Mise en place une hiérarchie de groupe
  - Mise en place de l'importation/exportation d'un formulaire
  - Transformation des pop up inutiles en notifications
  - Modification de l'interface de la page d'accueil
  - Ajout d'un menu d'actions multiples et modification de l'interface des actions applicables à 1 formulaire
  - Désactivation du bouton page suivante si des questions obligatoires n'ont pas encore de réponse
  - Ajout d'une option de duplication de composant

Afin de mettre en place la hiérarchie de groupes, de nombreuses modifications ont dû être apportées. Tout d'abord, il a fallu modifier la base de données en y ajoutant une table groups et une clé étrangère group_id dans la table formulaire faisant référence à l'id d'un groupe. Ensuite, nous avons créé les routes et services nécessaires aux groupes, ainsi qu'une route pour déplacer un formulaire dans un groupe (cela revient à lui modifier son attribut group_id).
Pour le côté front end, nous avons modifier l'interface en utilisant différentes vues (formulaire et groupes) avec un bouton permettant de naviguer entre les deux sans avoir à recharger la page (figure 8). Nous avons également ajouté une colonne affichant le nom du groupe auquel appartient un formulaire. S'il n'est dans aucun groupe, un "-" est affiché.

En plus de l'ajout des vue groupes et formulaires, la page d'accueil à subi d'autres changements d'interface. Nous avons regroupé les actions applicables aux formulaires dans un menu qui s'affiche au clique sur le bouton "..." présent dans la colonne action du tableau (figure 9). Il est aussi possible d'y accéder avec un clique droit sur la ligne du formulaire souhaité. Les mêmes opérations sont possibles sur les groupes, avec les actions applicables aux groupes. En bas de la page, nous avons ajouté un menu d'action permettant d'effectuer une action sur plusieurs formulaires/groupes à la fois. Toutes les actions sont possibles si un seul est sélectionné tandis que seules les actions groupées sont utilisables si plusieurs formulaires/groupes sont cochés (figure 8). Dans les deux vues, nous avons ajouté un filtre permettant de rechercher un formulaire ou un groupe par son nom. Enfin, uniquement dans la vue formulaire, nous avons ajouté un filtre permettant d'afficher les formulaires d'un groupe sélectionné (figure 8).

L'option "exporter" a été ajoutée dans les actions possibles pour un formulaire (figure 9).

Un bouton "Importer un formulaire" a également été rajouté. Ce bouton ouvre un modal (figure 13) qui permet d'importer un formulaire dans la base de données locale en y glissant un fichier json d'un formulaire préalablement exporté.

Nous avons remplacé le système de pop up existant par un système de notification qui apparaît en haut à droite de la page lorsqu'une action est effectuée (figure 12). Seule l'action de suppression affiche encore un pop up de confirmation.

Dans la page de création de formulaire, nous avons rajouté une fonctionnalité de duplication applicable sur les composants qui n'est pas intégrée dans FormJS (figure 10).

Enfin, si une page d'un formulaire contient un champ obligatoire, le bouton "page suivante" est désactivé et une indication visuelle est présente sur les champs obligatoires (figure 11).

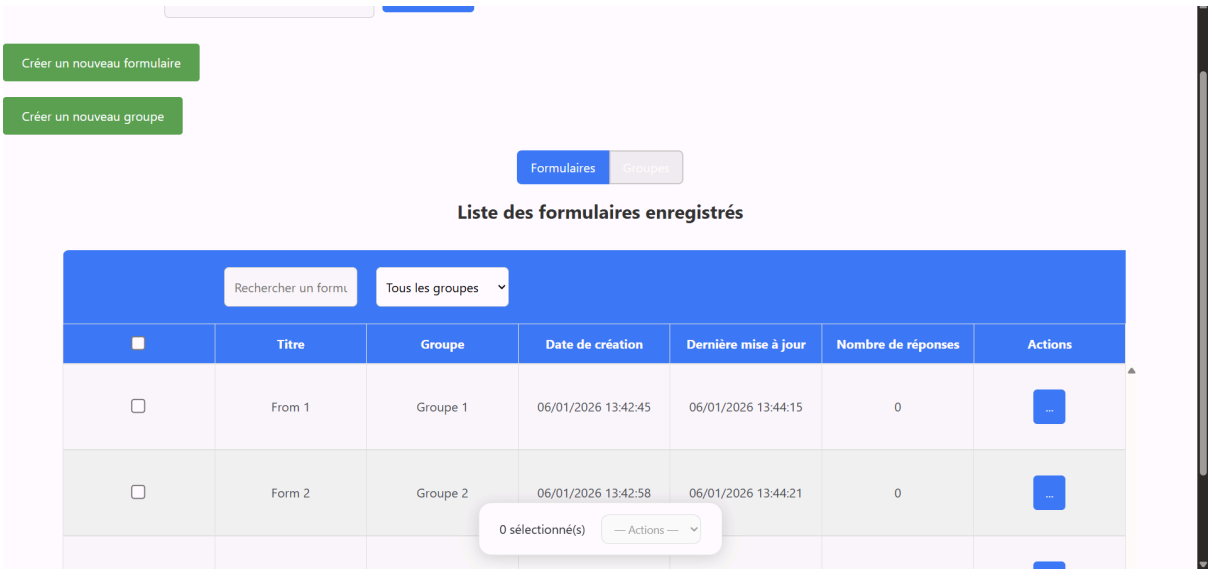


Figure 8 - Page d'accueil après itération 2

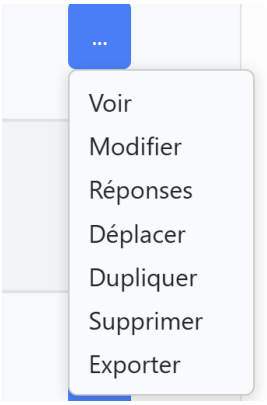


Figure 9 - Menu d'action sur un formulaire

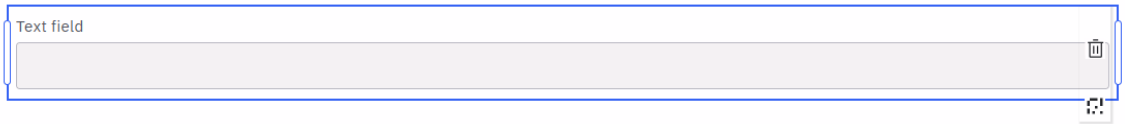


Figure 10 - Copier-coller des composants de formJS

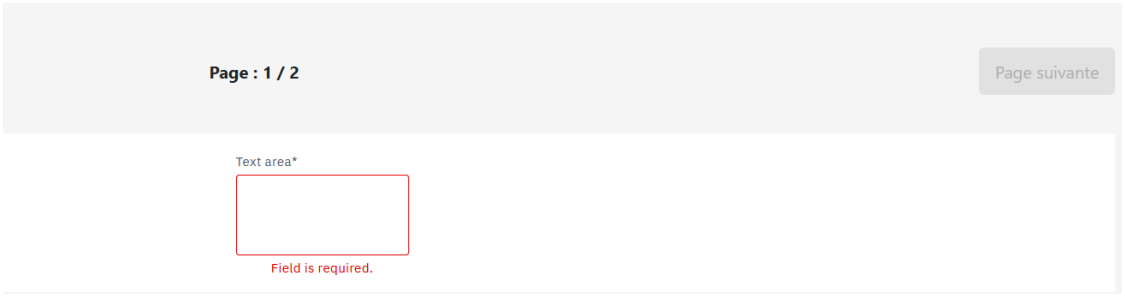


Figure 11 - Indication d'un champ obligatoire lorsqu'un utilisateur répond au questionnaire

---

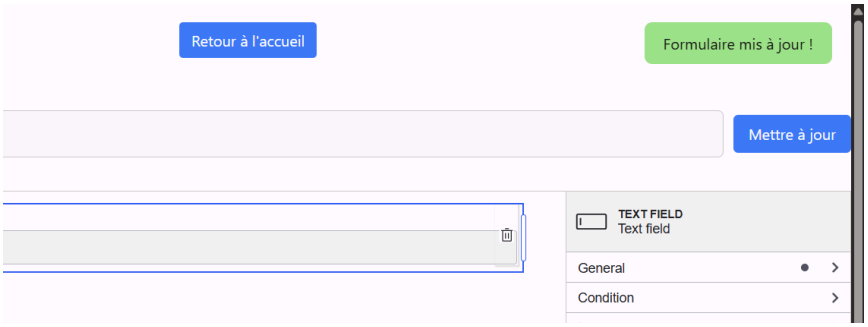


Figure 12 - Notification en haut à droite de la page au lieu d'un pop up (sauf pour la suppression)

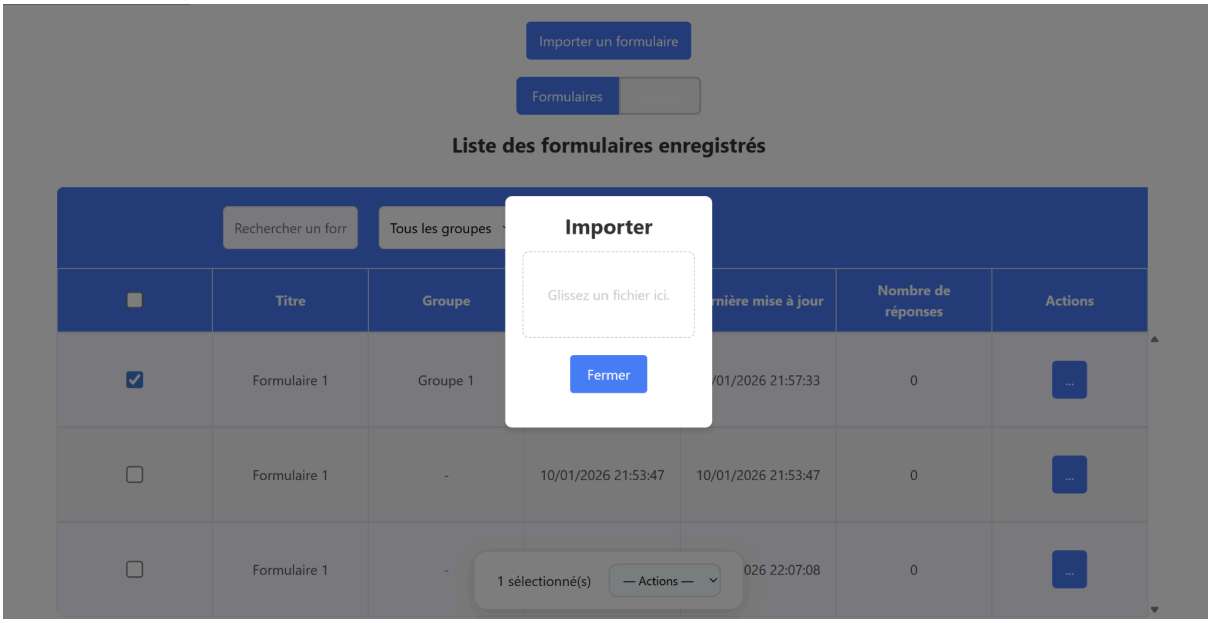


Figure 13 - Modal d'importation de formulaires.

- **Itération 3 -** Continuer à coder les tâches restantes :
    - Effectuer les modifications d'après les retours de l'itération précédente
    - Finaliser l'import/export pour les réponses et les groupes de formulaire

- Revoir l’architecture du fichier json d’exportation
- Ajouter une barre pour scroll le tableau des réponses
- Numéro de page en bas à droite
- Configurer le déploiement unique du client
- Restreindre la navigation à certaines pages spécifiques

Pour améliorer l’ergonomie de la consultation des réponses, une barre de défilement verticale a été ajoutée à la liste des réponses du formulaire. Concrètement, le tableau affichant les réponses est désormais contenu dans un bloc dont la hauteur maximale est limitée à 400 pixels. Lorsque le nombre de réponses dépasse cet espace, une barre de scroll apparaît automatiquement, permettant à l’utilisateur de faire défiler les données sans que la page entière ne soit allongée. Cette fonctionnalité a été implémentée en entourant le tableau d’un conteneur spécifique et en appliquant les propriétés CSS max-height: 400px et overflow-y: auto.

Nous avons ajouté une fonctionnalité permettant à l’administrateur de restreindre l’accès d’un utilisateur à un intervalle spécifique de pages d’un formulaire. Pour ce faire, l’administrateur saisit simplement l’intervalle souhaité directement dans l’URL du formulaire, par exemple :

http://localhost:3000/form-viewer/Form_0ecmxc3/1/2-6/u1?navigation=True

Dans cet exemple, le formulaire Form_0ecmxc3 est attribué à l’utilisateur u1, qui ne pourra parcourir que les pages 2 à 6. L’utilisateur n’a pas besoin de connaître la restriction : il voit uniquement la numérotation correspondant aux pages de l’intervalle sélectionné, de manière transparente (figure 15).

L’option d'exportation de formulaire a été modifiée. Désormais il est possible d’exporter les réponses des formulaires en plus du formulaire en lui-même. Pour cela, quand on va cliquer sur exporter, un modal apparaît à l’écran avec deux boutons, un pour exporter avec les réponses et l’autre non (figure 16). Il est également possible d’exporter plusieurs formulaires en une seule fois. Pour cela, il suffit de sélectionner tous les formulaires que l’on souhaite exporter puis cliquer sur le bouton exporter, un fichier json va alors être télécharger. Ce fichier contient tous les formulaires préalablement sélectionnés ainsi que leurs réponses si elles avaient été demandées.

L’importation de formulaire a également dû être modifiée pour suivre ces changements d’exportation. Déjà, il a fallu modifier l’API pour pouvoir importer le formulaire ainsi que ses réponses s’il y en a. Ensuite, cela a dû être adapté au nouveau format du json des formulaires exportés, ce qui permet alors d’importer, en une seule fois, autant de formulaires que souhaité.

Nous avons ajouté un script prod dans le fichier package.json afin de simplifier le déploiement de l’application. Ce script compile automatiquement le client puis démarre le serveur, permettant ainsi un lancement complet de l’application en production avec une seule commande (figure 16).



Figure 14 - barre de scroll sur le tableau des réponses

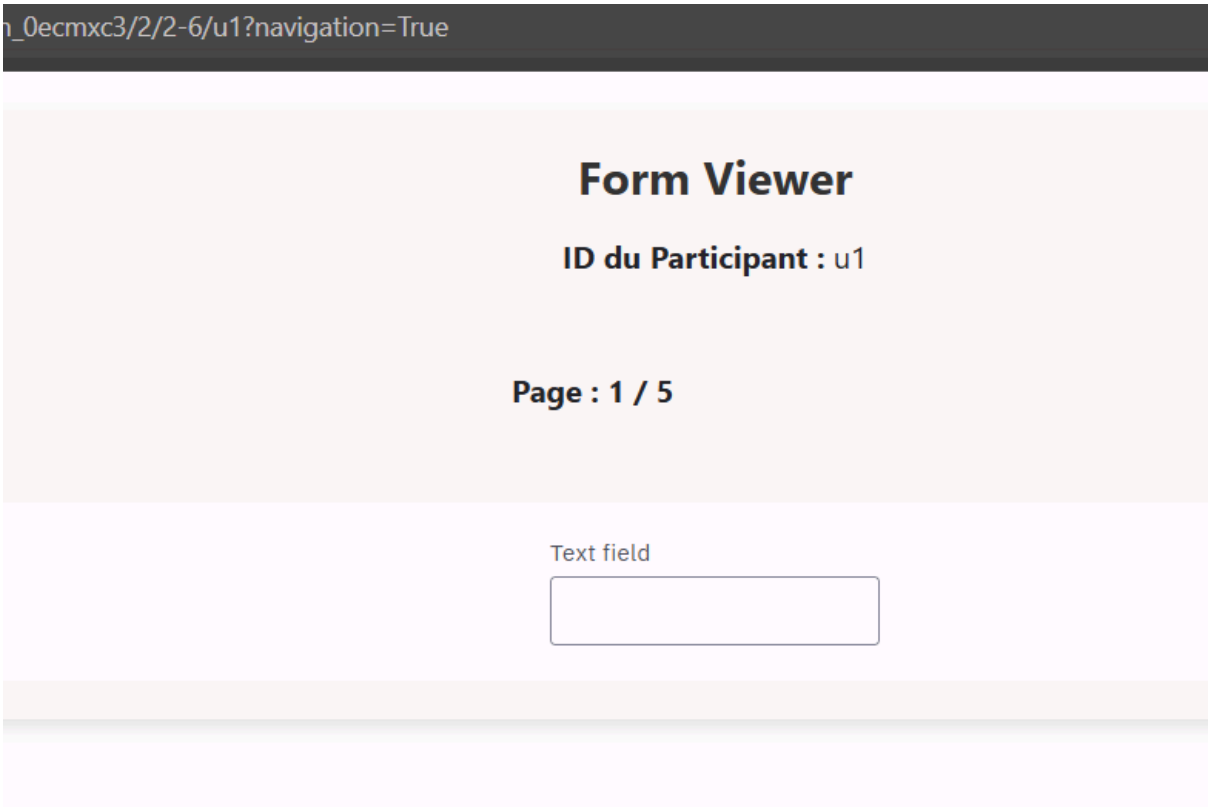


Figure 15 - Formulaire avec intervalle restreinte en vue utilisateur

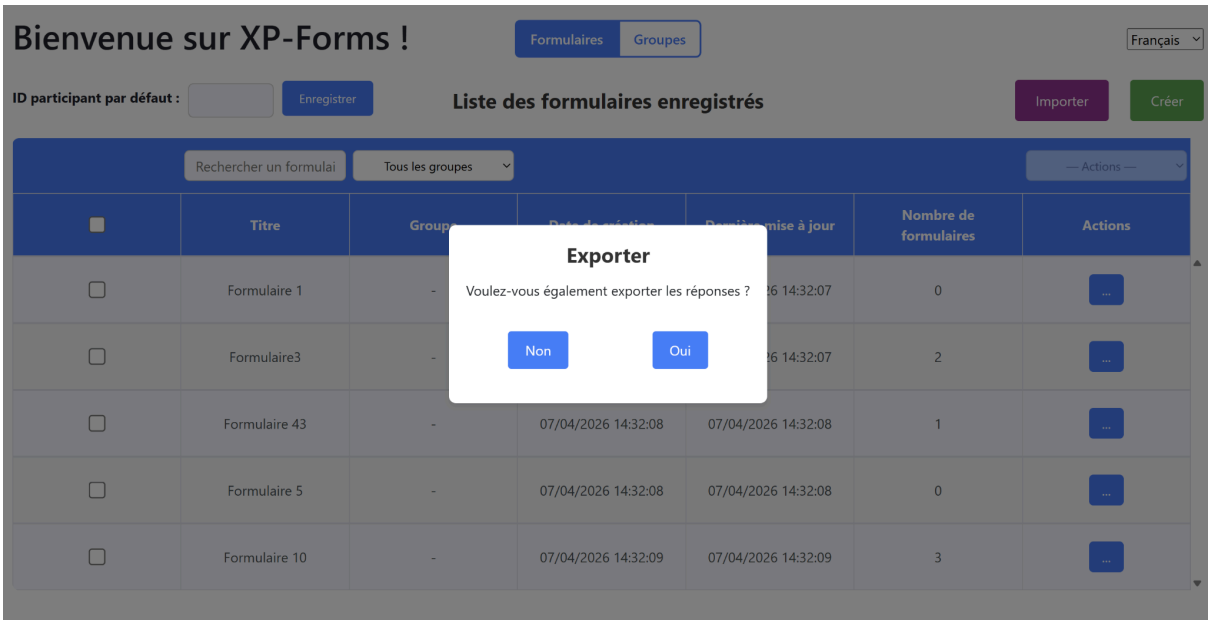


Figure 16 - Modal de choix d'export des réponses en plus du formulaire

```
▷Debug
"scripts": {
  "dev": "concurrently \"npm run server\" \"npm run client\"",
  "server": "cd serveur && npm run dev",
  "build": "cd client && npm run build",
  "client": "cd client && npm start",
  "start": "cd serveur && npm start",
  "prod": "npm run build && npm start",
  "install-all": "npm install && cd client && npm install && cd ../serveur && npm install"
}
}
```

Figure 17 - Scripts du package.json à la racine du projet

- **Itération 4**
  - Appliquer les modifications d'interface par rapport aux retours de l'itération précédente
  - L'internationalisation du site
  - Finir l'import/export, avec les retours de l'itération précédente
  - Se renseigner sur l'ajout de la possibilité d'effectuer des modifications de style sur les composants du formulaire

Le système de traduction (i18n) de l'application repose sur la bibliothèque i18next, associée à react-i18next pour l'intégration avec React. Les textes de l'interface sont centralisés dans des fichiers JSON distincts pour chaque langue, situés dans le dossier locales (par exemple, fr/translation.json pour le français et en/translation.json pour l'anglais). Lors de l'initialisation, le fichier i18n.js configure i18next en important ces ressources et en

définissant la langue par défaut ainsi qu'une langue de secours. Dans les composants React, le hook useTranslation permet d'accéder à la fonction t, qui sert à afficher les textes traduits en fonction de la langue active. Par exemple, pour traduire un bouton, il suffit d'utiliser {t("Back to home")} dans le code, et d'ajouter la clé correspondante dans chaque fichier de traduction.

Suite aux retours et aux usages, nous avons décidé de modifier la façon dont les formulaires sont exportés. En effet, mettre tous les formulaires exportés dans le même json n'est pas très pratique, la maintenabilité du fichier est laborieuse. Nous avons donc modifié cela pour que chaque formulaire exporté soit dans son propre fichier json avec le titre du formulaire en nom de fichier. Et tous ces json sont placés dans un fichier zip puis ce fichier est téléchargé.

L'importation a alors été modifiée en conséquence. Désormais il faut fournir un fichier zip contenant les json des formulaires pour l'importation ou alors, on peut donner uniquement un json de formulaire.

Nous avons étudié la possibilité de personnaliser les composants générés par FormJS en leur appliquant des styles spécifiques.Lors de cette phase d'exploration, nous avons constaté qu'il n'était pas possible d'injecter directement des classes CSS sur les composants. En effet, la bibliothèque régénère entièrement le DOM à chaque modification (changement de propriété, déplacement d'un élément, etc.). Par conséquent, toute modification manuelle du DOM (comme l'ajout de classes ou de styles) est perdue lors du prochain rendu.

Pour contourner cette contrainte, nous avons mis en place un mécanisme basé sur l'écoute de l'événement changed. À chaque mise à jour, les styles sont réappliqués dynamiquement en utilisant requestAnimationFrame, ce qui permet d'intervenir après le re-render effectué par la bibliothèque.

Par ailleurs, nous avons envisagé une approche plus directe consistant à intégrer des classes CSS directement dans le schéma des composants (par exemple via une propriété de type cssClass). Cependant, cette fonctionnalité n'est pas supportée nativement par FormJS, ce qui nous a amenés à concevoir une solution alternative.

Un prototype a été réalisé avec une fonctionnalité de mise en gras ("bold") :

- l'utilisateur sélectionne un composant,
- son identifiant est récupéré,
- un bouton permet d'ajouter la propriété bold = true dans l'attribut styles du schéma,
- à chaque mise à jour, le programme parcourt ces styles et les applique dynamiquement au composant concerné.

Cette approche permet de conserver les personnalisations malgré les re-rendus successifs, tout en s'intégrant au fonctionnement basé sur les schémas de FormJS.

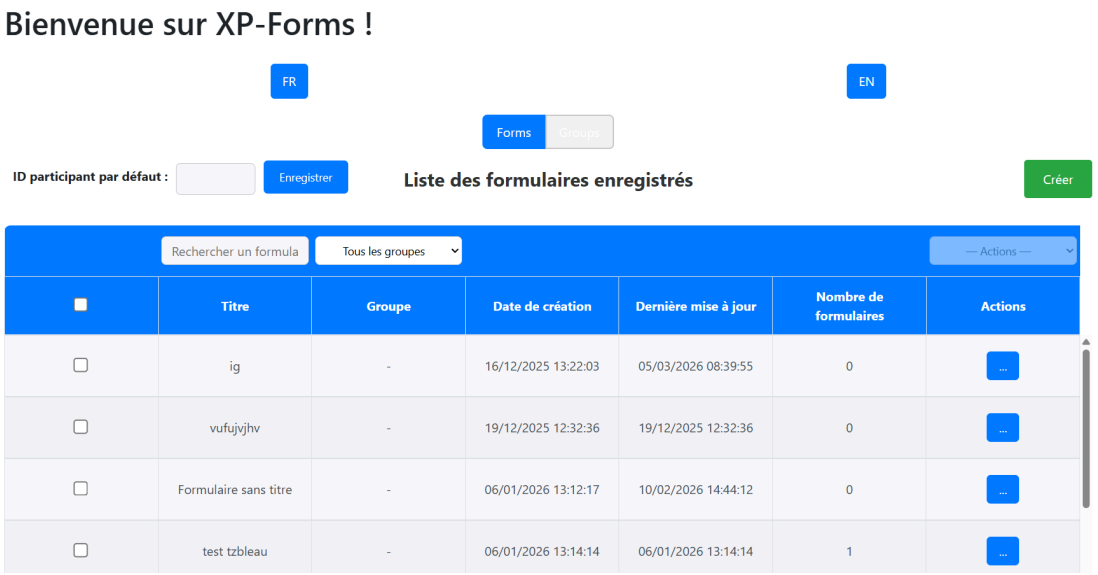


Figure 16 - Internationalisation du site (1)

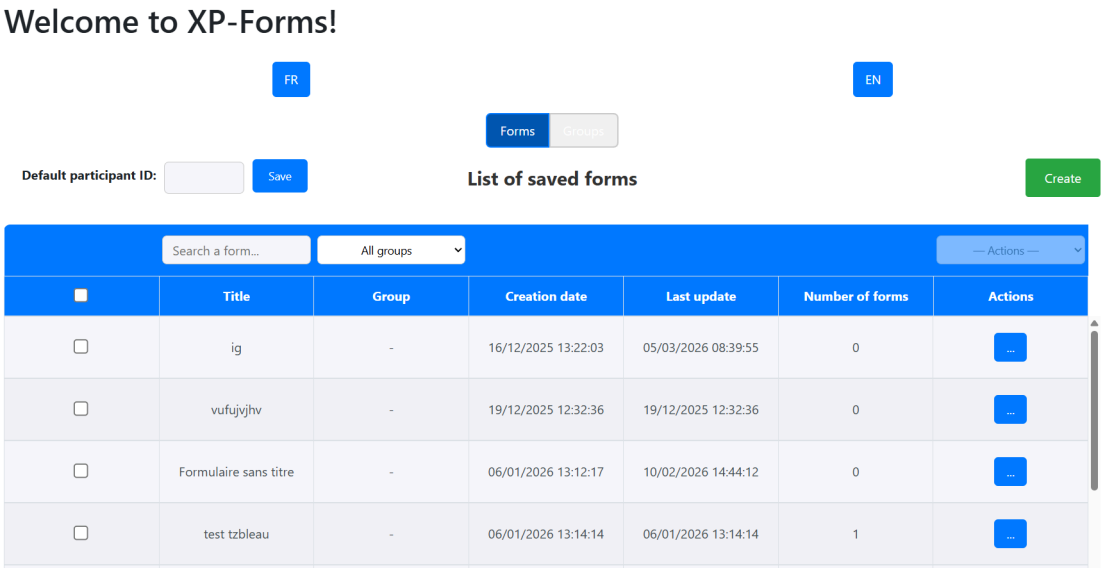


Figure 17 - Internationalisation du site (2)

- **Itération 5**
  - Modifier l'interface en fonction des remarques
  - Empêcher d'enregistrer un formulaire avec le même nom que qu'un autre formulaire avec message d'avertissement

- Formater la date dans les titres des exports de formulaires
- Ajouter notification d'erreur s'il y a une erreur lors de l'import d'un formulaire parmis d'autres qui s'importent
- Ajouter, dans la notification de succès des imports, le nombre de formulaires qui ont réussis à s'importer
- Vérifier si l'import/export fonctionne de la même manière peu importe la langue du pc
- Continuer de chercher des solutions pour les modifications de style sur les composants du formulaire
- Mettre les langues en haut à droite de la page d'accueil et les afficher les langues sous forme d'une liste déroulante
- Détecter la langue du navigateur pour sélectionner la langue affichée par défaut
- Mettre à jour toute la documentation

Suite à une précision des besoins, une fonctionnalité a été développée afin de permettre à l'utilisateur de saisir directement des propriétés de style dans une zone dédiée, puis de les appliquer aux composants du formulaire.

Lorsque l'utilisateur saisit des propriétés de style et valide leur application, le texte est transformé en objet JavaScript exploitable. Par exemple :

- bold: true devient { bold: true }
- color: #e74c3c devient { color: "#e74c3c" }
- fontSize: 18px devient { fontSize: "18px" }

Cet objet est ensuite ajouté directement dans le schéma du composant, sous une propriété styles. Les styles sont ainsi intégrés à la structure du formulaire et sauvegardés en base de données via l'API, ce qui garantit leur persistance.

Dans l'éditeur de FormJS, chaque composant rendu dans le DOM possède un attribut data-id, ce qui permet de cibler précisément les éléments correspondants. Un mécanisme de correspondance (mapping) est utilisé pour traduire les propriétés définies dans le schéma en propriétés CSS réelles. Certaines valeurs sont fixes (par exemple bold), tandis que d'autres sont dynamiques (comme color ou fontSize). Les styles sont ensuite appliqués dynamiquement aux éléments du DOM, en utilisant la priorité !important afin de surcharger les styles natifs de la bibliothèque. L'application concerne à la fois le conteneur du composant et certains éléments internes (labels, contenus textuels, etc.). Ce traitement est exécuté à chaque modification via l'événement changed, couplé à requestAnimationFrame, afin de garantir que les styles sont réappliqués après chaque re-rendu du DOM.

Contrairement à l'éditeur, le viewer de FormJS ne fournit pas d'attribut data-id, ce qui complique l'identification des composants dans le DOM. Pour résoudre ce problème, trois stratégies complémentaires ont été mises en place :

- **Identification via l'ID des inputs** : certains composants génèrent des éléments HTML dont l'identifiant contient celui du composant. Il est alors possible de les cibler via un sélecteur basé sur la fin de l'ID (figure 18).
- **Utilisation des attributs ARIA** : certains composants utilisent des attributs d'accessibilité (aria-labelledby, aria-describedby) contenant l'identifiant du composant, ce qui permet également de les retrouver.
- **Identification par type de composant** : pour les cas où aucune des méthodes précédentes ne fonctionne (par exemple pour les boutons radio ou les cases à cocher), les composants sont localisés via leur classe CSS spécifique, puis identifiés en comparant leur contenu textuel (figure 19).

Une fois le bon élément identifié, les styles sont appliqués de manière similaire à l'éditeur. Toutefois, afin d'éviter des effets indésirables liés à l'héritage CSS, certaines propriétés sont appliquées uniquement sur des éléments ciblés (comme les labels ou les contenus textuels), et non sur le conteneur principal.

Concernant l'exportation des formulaires, désormais le titre des zip est la date de l'export, au format YYYY-MM-DD suivi du nombre de formulaires exportés.

Pour l'importation, sur le modal, un input de type file été ajouté pour avoir un moyen plus simple que le drag and drop. Dorénavant, le drag and drop ne lance plus l'importation dès que l'on drop le zip dans la zone. Il y a un champ, en dessous de l'input, qui affiche le nom du fichier prêt à être importé, ce champ est commun et modifié par le drag and drop et le input. Un bouton "importer" a été ajouté en bas du modal pour importer le fichier sélectionné. (figure 21)

Une notification a été ajoutée pour indiquer s'il y a eu un problème avec un formulaire lors de l'importation. Cette notification indique le nom du formulaire ayant eu un problème.



Figure 18 - HTML dans l'inspection de l'aperçu d'un formulaire d'un composant "text field"

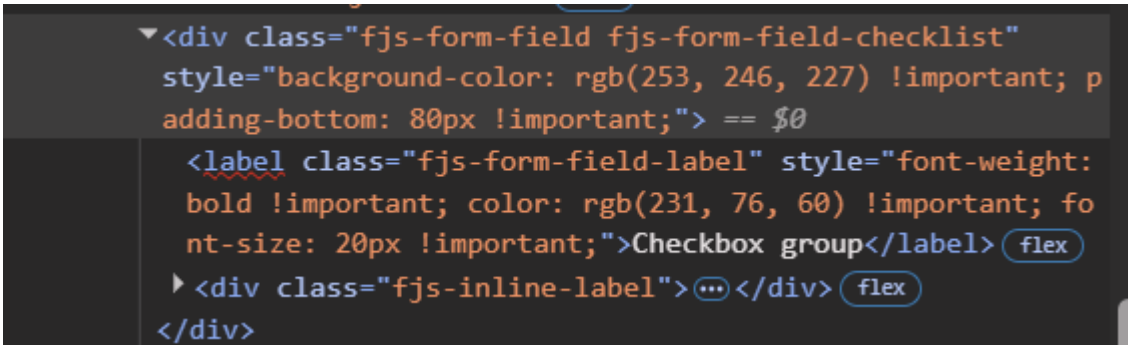


Figure 18 - HTML dans l'inspection de l'aperçu d'un formulaire d'un composant "checkbox group"

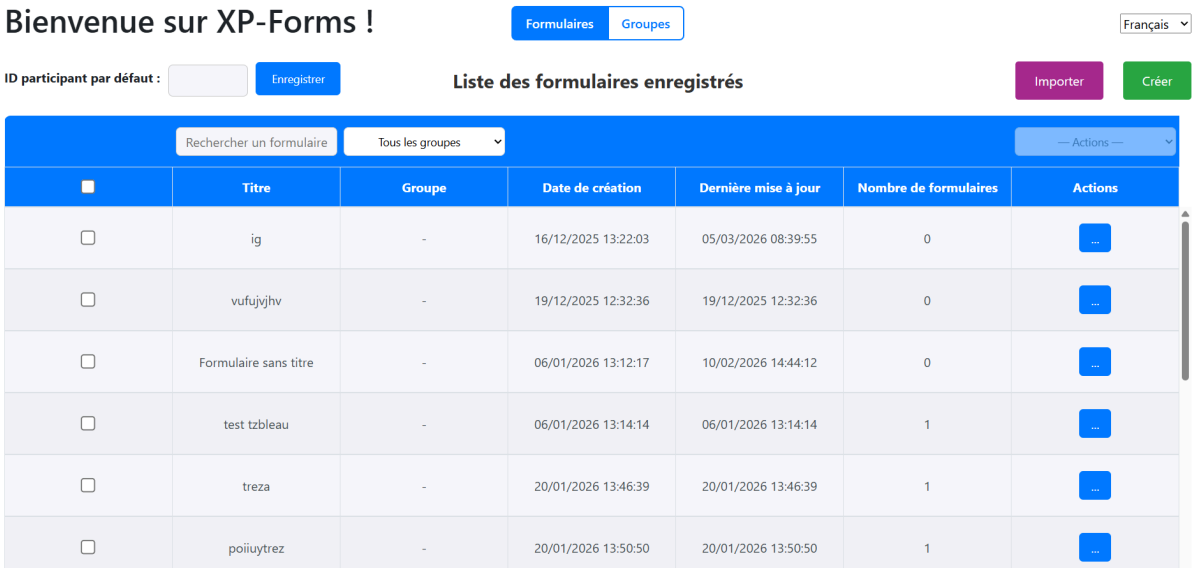


Figure 20 - langue liste déroulante, bouton import, et modification de l'interface pour qu'elle soit plus compacte

---

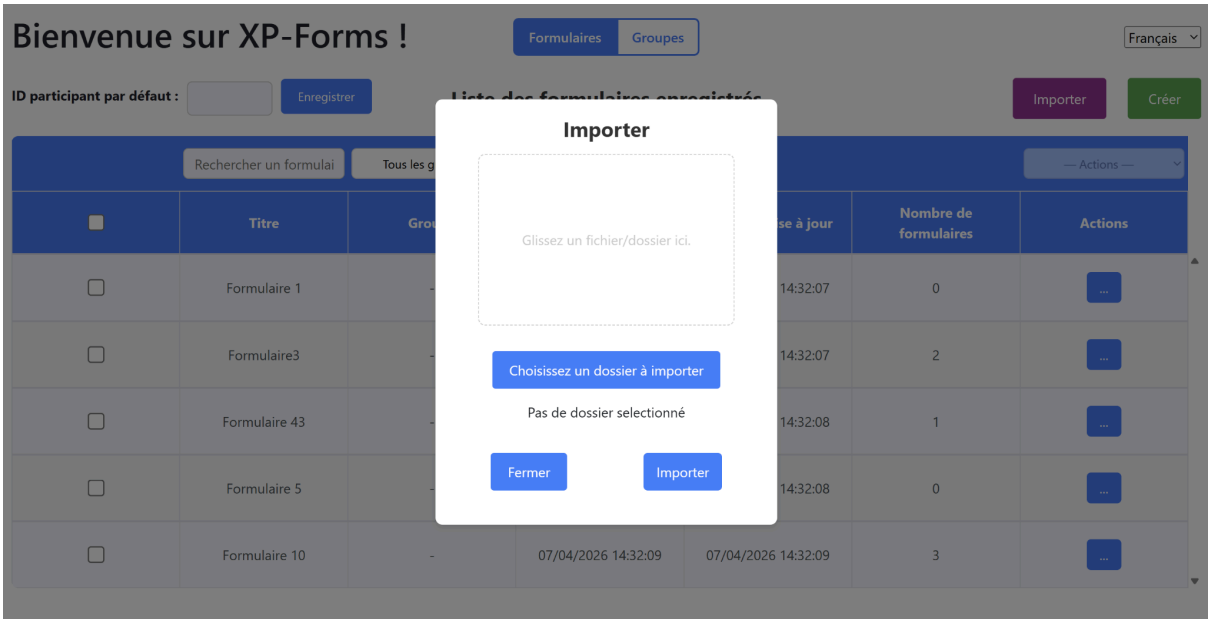


Figure 21 - Modal d'importation.



Figure 22 - Notifications d'erreur d'import de formulaire.

- **Prochains objectifs**
  - Ajouter de la cohérence dans le nom des composants css
  - Dans l'idéal pouvoir cibler titre du composant et son contenu séparément
  - Ajouter la possibilité d'utiliser tous les styles différents de css
  - Trouver un autre endroit ou pour le modificateur de css des composants (idéalement dans la même colonne que les paramètres
  - Détecter la langue de base du navigateur pour l'utiliser

- Dans le modal d’import : Modifier sélectionner un dossier en sélectionner un fichier
- Conserver message d'erreur lors de l’import pour indiquer d'où elle vient et la rendre accessible à l’utilisateur
- Mettre la traduction du message d’erreur pour créer un formulaire avec nom déjà existant
- Se renseigner sur la version de form js actuelle, et si une update est possible

Dans la continuité du travail réalisé sur la personnalisation des composants de FormJS, l’objectif est de faire évoluer le système initial basé sur une correspondance prédéfinie (mapping) entre des propriétés simplifiées et des styles CSS.

Cette première approche, reposant sur un ensemble limité de styles prédéfinis (pseudo-CSS), manque de flexibilité et nécessite des adaptations techniques pour chaque nouvelle propriété à supporter.

L’évolution prévue vise à permettre à l’utilisateur de saisir directement du CSS dans un panneau dédié. Les styles seront alors interprétés et appliqués tels quels aux composants, sans passer par une couche intermédiaire de transformation.

Cette approche offrirait une plus grande liberté de personnalisation, tout en simplifiant l’architecture du système en supprimant la dépendance à une liste de styles prédéfinis.

Répartition des tâches

Tâche	Pablo	Elouan	Mathis
Gestion de l’export			
Duplication de composants			
Gestion des groupes			
Gestion de l’import			
Internationalisation			
Modification du CSS des composants			
Modifications de l’interface			

3 - Revue de littérature scientifique et technologique

Expérience utilisateur et qualité des données expérimentales

L'expérience utilisateur (UX) joue un rôle central dans la qualité des données collectées lors d'expériences scientifiques impliquant des interfaces numériques. Une interface mal conçue peut introduire des biais, augmenter la charge cognitive des participants et affecter directement la précision des réponses fournies. À l'inverse, une interface claire, cohérente et ergonomique favorise l'engagement des participants et améliore la fiabilité des données recueillies (de Leeuw et al., 2021).

Plusieurs travaux en sciences cognitives et en interaction homme-machine montrent que la charge cognitive induite par une interface influence la performance des utilisateurs. Une surcharge cognitive peut entraîner des erreurs de compréhension, des réponses approximatives ou des abandons prématurés des questionnaires. Holmqvist et al. (2011) soulignent que des éléments visuels mal positionnés ou des interactions peu intuitives augmentent le temps de fixation visuelle et traduisent une difficulté de traitement de l'information par l'utilisateur.

Dans le cadre de questionnaires expérimentaux, ces enjeux sont particulièrement critiques. Les participants doivent se concentrer sur les stimuli présentés et sur les consignes de l'expérience, sans être distraits par des problèmes d'interface. Une UX de qualité contribue ainsi à réduire les biais expérimentaux et à garantir que les réponses collectées reflètent réellement les phénomènes étudiés plutôt que des difficultés d'interaction avec l'outil (de Leeuw et al., 2021).

Conception de formulaires interactifs en contexte expérimental

Les formulaires interactifs constituent un outil central dans de nombreuses expériences comportementales, qu'ils soient utilisés en pré-expérience, post-expérience ou intégrés directement au protocole expérimental. Contrairement aux formulaires classiques destinés à un usage administratif, les formulaires expérimentaux doivent répondre à des exigences spécifiques en termes de précision, de contrôle et de reproductibilité.

La structuration des formulaires, notamment la pagination, la validation conditionnelle des réponses et l'ordre d'apparition des questions, peut influencer les stratégies de réponse des participants. Une mauvaise organisation peut provoquer des réponses mécaniques ou un désengagement progressif, réduisant ainsi la qualité des données collectées. De Leeuw et al. (2021) montrent que les plateformes numériques bien conçues permettent de limiter ces effets en automatisant les contrôles de validité et en guidant les participants de manière fluide tout au long du questionnaire.

L'intégration de fonctionnalités avancées, telles que la conditionnalité des questions ou la duplication de composants, permet également d'adapter les formulaires à des contextes expérimentaux variés. Ces fonctionnalités facilitent la création de protocoles complexes tout

en maintenant une cohérence visuelle et fonctionnelle, essentielle pour limiter les biais liés à l'interface.

Oculométrie et analyse de l'interaction utilisateur

L'oculométrie (eye-tracking) est une méthode d'analyse comportementale largement utilisée pour étudier l'attention visuelle et les stratégies de lecture des utilisateurs lors de l'interaction avec une interface numérique. Elle repose sur l'enregistrement précis des mouvements oculaires, tels que les fixations et les saccades, afin de comprendre comment les utilisateurs perçoivent et traitent l'information (Duchowski, 2022).

Les données oculométriques permettent d'identifier les zones d'une interface qui attirent le plus l'attention, celles qui sont ignorées, ainsi que les éléments qui génèrent une surcharge cognitive. Par exemple, des fixations prolongées sur un bouton ou un texte peuvent indiquer une ambiguïté ou une difficulté de compréhension (Holmqvist et al., 2011). Ces informations sont particulièrement utiles pour évaluer l'ergonomie des formulaires interactifs et optimiser leur conception.

Dans le contexte de la recherche expérimentale, l'oculométrie constitue un outil précieux pour analyser l'impact de l'interface sur le comportement des participants. L'intégration de solutions telles que Tobii Pro Lab permet de synchroniser les données oculométriques avec les réponses aux questionnaires, offrant ainsi une analyse plus fine des interactions et des biais potentiels induits par l'interface (Duchowski, 2022).

Collecte de données numériques et biais méthodologiques

La collecte de données expérimentales via des plateformes numériques présente de nombreux avantages, notamment en termes de flexibilité, d'automatisation et de capacité à toucher un large échantillon de participants. Cependant, elle soulève également des enjeux méthodologiques importants liés à la fiabilité et à la comparabilité des données.

De Leeuw et al. (2021) comparent la qualité des données issues d'expériences en ligne et en laboratoire, et montrent que, bien que les résultats soient globalement comparables, des biais spécifiques peuvent apparaître en raison de problèmes techniques, d'environnements non contrôlés ou d'interfaces inadaptées. Ces biais peuvent se traduire par des données manquantes, des réponses incohérentes ou une variabilité accrue des résultats.

La conception d'outils robustes, intégrant des mécanismes de validation, de suivi et de monitoring des réponses, est donc essentielle pour garantir la qualité des données collectées. Une attention particulière doit être portée à la stabilité technique du système, afin d'éviter toute perte de données lors de la collecte, ainsi qu'à la clarté des consignes présentées aux participants.

Sécurité, éthique et protection des données expérimentales

La collecte de données expérimentales implique des responsabilités éthiques et juridiques importantes, en particulier lorsque les données sont recueillies via des plateformes numériques. En Europe, ces pratiques sont encadrées par le Règlement Général sur la Protection des Données (RGPD), qui impose des principes tels que la minimisation des données, la transparence et la protection des droits des participants (European Union, 2016).

Les chercheurs et développeurs doivent s'assurer que seules les données strictement nécessaires à l'analyse scientifique sont collectées, et que celles-ci sont stockées de manière sécurisée. L'anonymisation ou la pseudonymisation des données constitue un élément clé pour réduire les risques liés à la protection de la vie privée des participants.

Dans le cadre des outils expérimentaux, le respect du RGPD ne se limite pas à une contrainte légale, mais participe également à la qualité scientifique des études. Une gestion rigoureuse des données renforce la confiance des participants et contribue à la reproductibilité des expériences (European Union, 2016).

Logiciels open source et recherche expérimentale

L'utilisation de logiciels open source dans le domaine de la recherche expérimentale présente de nombreux avantages, notamment en termes de transparence, de reproductibilité et de collaboration scientifique. Les outils open source permettent aux chercheurs de comprendre, modifier et adapter les logiciels à leurs besoins spécifiques, favorisant ainsi l'innovation et la diffusion des connaissances.

Cependant, le recours à des solutions open source implique également des défis, notamment en matière de maintenance, de sécurité et de dépendance à des bibliothèques tierces. Une veille technologique régulière et une documentation claire sont nécessaires pour garantir la pérennité des outils développés.

Dans ce contexte, le développement d'outils expérimentaux open source, compatibles avec des plateformes telles que OpenSesame ou Tobii Pro Lab, s'inscrit dans une démarche de science ouverte visant à mutualiser les ressources et à améliorer la qualité des dispositifs expérimentaux.

Architectures web modernes pour applications expérimentales

Le développement d'applications web dédiées à la recherche expérimentale repose sur des architectures capables de gérer des interactions complexes, un volume potentiellement important de données et des exigences élevées en matière de stabilité et de maintenabilité. Les architectures dites full-stack JavaScript, combinant un front-end en React et un back-end

basé sur Node.js, sont aujourd'hui largement adoptées pour ce type d'applications en raison de leur flexibilité et de leur cohérence technologique (Tilkov & Vinoski, 2010).

Node.js permet d'exécuter du JavaScript côté serveur et repose sur un modèle événementiel non bloquant, particulièrement adapté aux applications nécessitant de nombreuses requêtes concurrentes, comme les plateformes de collecte de données expérimentales. Le framework Express.js, construit au-dessus de Node.js, fournit une structure légère pour la création d'API REST, facilitant la gestion des routes, des requêtes HTTP et des middlewares (Express.js Foundation, 2024).

Pour des projets de plus grande envergure, des frameworks plus structurés tels que NestJS proposent une architecture modulaire inspirée des principes de l'ingénierie logicielle (injection de dépendances, séparation des responsabilités, tests unitaires). NestJS s'appuie sur Express ou Fastify et permet d'améliorer la maintenabilité et l'évolutivité du code, ce qui constitue un enjeu majeur dans des projets amenés à évoluer sur plusieurs années et à être repris par différentes équipes (NestJS Documentation, 2024).

Dans le contexte d'une application expérimentale open source, ces choix technologiques influencent directement la capacité du projet à évoluer, à intégrer de nouvelles fonctionnalités et à garantir une stabilité suffisante lors des phases de collecte de données.

Outils de développement collaboratif et gestion des versions

La gestion des versions constitue un élément central du développement logiciel, en particulier dans des projets collaboratifs et évolutifs comme les outils de recherche expérimentale. Les systèmes de contrôle de version permettent de conserver un historique précis des modifications, de travailler en parallèle sur différentes fonctionnalités et de garantir la traçabilité des évolutions techniques.

Git est aujourd'hui le système de gestion de versions distribué le plus utilisé. Il permet à chaque développeur de disposer d'une copie complète du dépôt et de gérer localement les modifications avant de les intégrer à la branche principale. Cette approche favorise l'expérimentation et limite les risques de perte de code (Chacon & Straub, 2014).

GitHub, en tant que plateforme collaborative, enrichit Git par des fonctionnalités telles que la gestion des branches, les pull-requests et le suivi des issues. Ces outils facilitent la collaboration entre développeurs, la revue de code et la documentation des choix techniques. Dans un contexte académique et open source, GitHub joue également un rôle important dans la pérennité des projets, en permettant à de nouvelles équipes de reprendre et de prolonger un travail existant, comme c'est le cas pour ce projet (Chacon & Straub, 2014).

L'utilisation combinée de Git et GitHub contribue ainsi à améliorer la qualité du code, la communication au sein de l'équipe et la reproductibilité des développements, éléments essentiels dans un projet de recherche.

Internationalisation des interfaces (i18n)

L'internationalisation des interfaces numériques, souvent appelée i18n (abréviation de internationalisation), consiste à concevoir un logiciel de manière à pouvoir l'adapter facilement à différentes langues. Concrètement, cela signifie que les textes affichés dans l'interface sont séparés du code de l'application afin de pouvoir être traduits sans modifier la logique du programme.

L'internationalisation repose généralement sur l'utilisation de fichiers de traduction contenant l'ensemble des textes utilisés dans l'interface. Chaque langue possède son propre fichier, et l'application affiche les textes correspondants selon la langue sélectionnée. Cette organisation permet de modifier ou d'ajouter des traductions sans avoir à modifier directement le code de l'application.

Dans les applications web modernes, l'internationalisation est souvent mise en place à l'aide de bibliothèques spécialisées qui facilitent la gestion des traductions et leur intégration dans l'interface. Cette approche permet de structurer le projet de manière plus claire, de centraliser les textes utilisés dans l'application et de simplifier la maintenance du code lorsque plusieurs langues doivent être prises en charge.

4 - Bibliographie

Duchowski, A. T. (2022). Eye Tracking Methodology: Theory and Practice (3rd ed.). Springer. https://doi.org/10.1007/978-3-030-83429-7

Holmqvist, K., Nyström, M., Andersson, R., Dewhurst, R., Jarodzka, H., & van de Weijer, J. (2011). Eye Tracking: A Comprehensive Guide to Methods and Measures. Oxford University Press.

de Leeuw, J. R., Motz, B. A., Psy, D., & McDermott, K. B. (2021). Comparing data quality from an online and in-person lab sample on dynamic theory of mind tasks. Behavior Research Methods, 53(3), 1006–1021. https://doi.org/10.3758/s13428-020-01438-2

European Union. (2016). Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation). https://eur-lex.europa.eu/eli/reg/2016/679/oj

Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to build high-performance network programs. IEEE Internet Computing, 14(6), 80–83. https://doi.org/10.1109/MIC.2010.145

Express.js Foundation. (2024). Express – Fast, unopinionated, minimalist web framework for Node.js. https://expressjs.com/

NestJS. (2024). NestJS – A progressive Node.js framework. https://nestjs.com/

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/en/v2

Facebook Open Source. (2024). React – A JavaScript library for building user interfaces. https://react.dev/

BPMN.io. (2024). Form-js Documentation. https://form-js.dev/

Documentation de i18n : https://www.i18next.com/

5 - Conclusion

À l'issue de ce rapport intermédiaire, les premières itérations du projet ont permis de poser des bases solides pour l'amélioration du site. L'analyse des besoins et la modélisation du domaine ont permis de traduire les attentes de la maîtrise d'ouvrage en choix techniques cohérents et réalistes.

Grâce à une organisation en itérations et à une démarche Agile, l'équipe a pu prendre en main l'architecture existante et implémenter plusieurs fonctionnalités essentielles visant à améliorer l'ergonomie, la gestion des formulaires et le suivi des réponses.

La suite du projet consistera à finaliser les fonctionnalités restantes, consolider les mécanismes d'import/export et améliorer la stabilité et la maintenabilité de la plateforme, en vue de la soutenance finale et de la livraison d'un outil fiable pour le XP-LAB.